Университет ИТМО
Факультет программной инженерии и компьютерной техники

Группа: P3311
Студент: Горошников Т.И.
Преподаватель: Потемин И.С.

Отчет по лабораторной работе №2 “Рассчет интеграла функции на заданном промежутке методом Монте-Карло“

Цель работы

Изучить базовые методы Монте-Карло, используемые для интегрирования функций на заданном интервале, и оценить точность интегрирования и основные преимущества и недостатки этих методов.

Используемые формулы

1. Простое интегрирование методом Монте-Карло:

$$I = \frac{b-a}{n} \sum_{i=0}^{n} f(x_i)$$

2. Интегрирование методом Монте-Карло со стратификацией:
   Пусть $j = \overline{1, m}$ — номер отрезка разбиения, l — длина отрезка разбиения.
   Тогда:

$$I = \sum_{j=1}^{m} \frac{l}{n} \sum_{i=0}^{n} f(x_{j_i})$$

3. Интегрирование методом Монте-Карло с выборкой по значимости:

$$I = \frac{1}{n} \sum_{i=0}^{n} \frac{f(x_i)}{p(x_i)}$$

4. Интегрирование методом Монте-Карло с многократной выборкой по значимости:

$$I = \frac{1}{n} \sum_{i=0}^{n} f(x_i) \left(\frac{p_1(x_i)}{w_1(x_i)} + \frac{p_2(x_i)}{w_2(x_i)} \right)$$

5. Интегрирование методом Монте-Карло с использованием русской рулетки:

$$I = \frac{1}{n} \sum_{i=0}^{n} \frac{f(x_i)}{p(x_i) \cdot R}$$

Программа для рассчета интегралов

Весь код можно также найти по ссылке.
Фрагмент файла `main.py` с основными вычислениями:

```
rng = np.random.default_rng(35)
```

```
def calculate_simple_monte_carlo_integral(a, b, function, n):

    samples = rng.uniform(a, b, size=n)

    y = function(samples)

    return np.mean(y) * (b - a)



def calculate_stratified_monte_carlo_integral(a, b, function, step, n):

    partition = np.arange(a, b + 1e-6, step)

    result = 0

    for i in range(len(partition) - 1):

        samples = rng.uniform(partition[i], partition[i + 1], size=n)

        result += np.mean(function(samples)) * (partition[i + 1] -
partition[i])

    return result



def calculate_importance_monte_carlo_integral(function, pdf, inv_cdf,
n):

    samples = inv_cdf(rng.uniform(0, 1, size=n))

    y = function(samples) / pdf(samples)

    return np.mean(y)



def calculate_multi_importance_monte_carlo_integral(function, pdfs,
inv_cdfs, weight_functions, n):

    base_samples = rng.uniform(0, 1, size=n)
```

```
    result = 0

    for pdf, inv_cdf, weight_function in zip(pdfs, inv_cdfs,
weight_functions):

        samples = inv_cdf(base_samples)

        result += function(samples) / pdf(samples) *
weight_function(samples, pdfs[0], pdfs[1])

    return np.mean(result)



def calculate_russian_roulette_monte_carlo_integral(function, pdf,
inv_cdf, r, n):

    y = 0

    for _ in range(n):

        sample = inv_cdf(rng.uniform(0, 1))

        if rng.uniform(0, 1) < r:

            y += function(sample) / (pdf(sample) * r)

    return y / n




square = lambda x: x ** 2

square_integral = lambda x: x ** 3 / 3



p1 = lambda x: x * 2 / 21

p2 = lambda x: x ** 2 / 39
```

```
p3 = lambda x: x ** 3 * 4 / 609



f1 = lambda x: (x * 21 + 4) ** (1 / 2)

f2 = lambda x: (x * 117 + 8) ** (1 / 3)

f3 = lambda x: (x * 609 + 16) ** (1 / 4)



w1 = lambda x, pdf1, pdf2: pdf1(x) / (pdf1(x) + pdf2(x))

w2 = lambda x, pdf1, pdf2: pdf2(x) / (pdf1(x) + pdf2(x))

w3 = lambda x, pdf1, pdf2: pdf1(x) ** 2 / (pdf1(x) ** 2 + pdf2(x) ** 2)

w4 = lambda x, pdf1, pdf2: pdf2(x) ** 2 / (pdf1(x) ** 2 + pdf2(x) ** 2)



r1 = 0.5

r2 = 0.75

r3 = 0.95



n1 = 100

n2 = 1000

n3 = 10000

n4 = 100000



a, b = 2, 5
```

```
true_integral_value = square_integral(5) - square_integral(2)




def run_experiments(a, b, true_integral_value):

    results = []



    for n in [n1, n2, n3, n4]:

        estimate = calculate_simple_monte_carlo_integral(a, b, square,
n)

        results.append(

            {

                "method": "simple",

                "n": n,

                "params": "-",

                "estimate": estimate,

                "abs_error": abs(estimate - true_integral_value),

            }

        )



        for step in [1, 0.5]:

            estimate = calculate_stratified_monte_carlo_integral(a, b,
square, step, n)

            results.append(
```

```
                {
                    "method": "stratified",
                    "n": n,
                    "params": f"step={step}",
                    "estimate": estimate,
                    "abs_error": abs(estimate - true_integral_value),
                }
            )


        for pdf_name, pdf, inv_cdf in zip(["p1", "p2", "p3"], [p1, p2, p3], [f1, f2, f3]):
            estimate = calculate_importance_monte_carlo_integral(square, pdf, inv_cdf, n)
            results.append(
                {
                    "method": "importance",
                    "n": n,
                    "params": f"pdf={pdf_name}",
                    "estimate": estimate,
                    "abs_error": abs(estimate - true_integral_value),
                }
            )
```

```
            for r in [r1, r2, r3]:

                estimate =
calculate_russian_roulette_monte_carlo_integral(square, pdf, inv_cdf, r,
n)

                results.append(

                    {

                        "method": "russian_roulette",

                        "n": n,

                        "params": f"pdf={pdf_name}, r={r}",

                        "estimate": estimate,

                        "abs_error": abs(estimate -
true_integral_value),

                    }

                )



        for weight_name, weight_function_pair in [("balance", [w1, w2]),
("power", [w3, w4])]:

            estimate = calculate_multi_importance_monte_carlo_integral(

                square, [p1, p3], [f1, f3], weight_function_pair, n

            )

            results.append(

                {

                    "method": "multi_importance",
```

```
                    "n": n,
                    "params": f"pdfs=p1,p3; weights={weight_name}",
                    "estimate": estimate,
                    "abs_error": abs(estimate - true_integral_value),
                }
            )

    return results
```

Результаты вычислений

Истинное значение интеграла:

$$I = \int_2^5 x^2 dx = \frac{x^3}{3}|_2^5 = \frac{125-8}{3} = 39$$

n	params	estimate	abs_error
100	-	37.7766	1.22343
1000	-	37.9526	1.0474
10000	-	38.9943	0.005676
100000	-	38.9796	0.020392

Результаты простого интегрирования методом Монте-Карло

n	params	estimate	abs_error
100	step=0.5	38.9305	0.069517
100	step=1	38.8105	0.189512
1000	step=0.5	39.0267	0.026703
1000	step=1	38.8825	0.117538
10000	step=0.5	39.0037	0.003674
10000	step=1	39.0022	0.002176
100000	step=0.5	38.9913	0.008701
100000	step=1	39.0109	0.010925

Результаты интегрирования методом Монте-Карло со стратификацией

n	params	estimate	abs_error
100	pdf=p1	39.0836	0.083609
100	pdf=p2	39	0
100	pdf=p3	38.2425	0.75748
1000	pdf=p1	39.1475	0.147459
1000	pdf=p2	39	0
1000	pdf=p3	39.0798	0.079787
10000	pdf=p1	39.1001	0.10007
10000	pdf=p2	39	0
10000	pdf=p3	39.0594	0.05943
100000	pdf=p1	39.0239	0.023855
100000	pdf=p2	39	0
100000	pdf=p3	39.0411	0.041064

Результаты интегрирования методом Монте-Карло с выборкой по значимости

n	params	estimate	abs_error
100	pdfs=p1,p3; weights=balance	39	3e-06
100	pdfs=p1,p3; weights=power	38.8731	0.126904
1000	pdfs=p1,p3; weights=balance	39.0149	0.0149
1000	pdfs=p1,p3; weights=power	39.0751	0.07511
10000	pdfs=p1,p3; weights=balance	39.0128	0.012812
10000	pdfs=p1,p3; weights=power	39.009	0.009038
100000	pdfs=p1,p3; weights=balance	38.9971	0.00288
100000	pdfs=p1,p3; weights=power	39.0157	0.015696

Результаты интегрирования методом Монте-Карло с многократной выборкой по значимости

n	params	estimate	abs_error
100	pdf=p1, r=0.5	35.1438	3.85623
100	pdf=p1, r=0.75	39.9578	0.957789
100	pdf=p1, r=0.95	40.3128	1.31278
100	pdf=p2, r=0.5	36.66	2.34
100	pdf=p2, r=0.75	41.08	2.08
100	pdf=p2, r=0.95	39.8211	0.821053
100	pdf=p3, r=0.5	31.0145	7.98547
100	pdf=p3, r=0.75	39.1173	0.117258
100	pdf=p3, r=0.95	38.5842	0.415776
1000	pdf=p1, r=0.5	37.1933	1.80666
1000	pdf=p1, r=0.75	39.1032	0.103174
1000	pdf=p1, r=0.95	38.5859	0.414101
1000	pdf=p2, r=0.5	37.05	1.95
1000	pdf=p2, r=0.75	38.272	0.728
1000	pdf=p2, r=0.95	38.8358	0.164211
1000	pdf=p3, r=0.5	39.8075	0.807456
1000	pdf=p3, r=0.75	38.7721	0.227932
1000	pdf=p3, r=0.95	38.8029	0.197086

Результаты интегрирования методом Монте-Карло с использованием русской рулетки (1/2)

10000	pdf=p1, r=0.5	39.039	0.039034
10000	pdf=p1, r=0.75	39.2688	0.26879
10000	pdf=p1, r=0.95	38.9857	0.014301
10000	pdf=p2, r=0.5	38.8206	0.1794
10000	pdf=p2, r=0.75	38.8752	0.1248
10000	pdf=p2, r=0.95	38.9795	0.020526
10000	pdf=p3, r=0.5	39.2801	0.280144
10000	pdf=p3, r=0.75	39.1418	0.141754
10000	pdf=p3, r=0.95	38.9322	0.067831
100000	pdf=p1, r=0.5	39.1164	0.116351
100000	pdf=p1, r=0.75	39.0751	0.075064
100000	pdf=p1, r=0.95	38.9824	0.017581
100000	pdf=p2, r=0.5	39.0421	0.04212
100000	pdf=p2, r=0.75	39.0286	0.0286
100000	pdf=p2, r=0.95	38.9762	0.023811
100000	pdf=p3, r=0.5	38.9532	0.046759
100000	pdf=p3, r=0.75	38.9309	0.069064
100000	pdf=p3, r=0.95	38.9892	0.010802

Результаты интегрирования методом Монте-Карло с использованием русской рулетки (2/2)

method	n	params	estimate	abs_error
simple	10000	-	38.9943	0.005676
stratified	10000	step=1	39.0022	0.002176
importance	100	pdf=p2	39	0
multi_importance	100	pdfs=p1,p3; weights=balance	39	3e-06
russian_roulette	100000	pdf=p3, r=0.95	38.9892	0.010802

Лучшие результаты от каждого метода

Вывод

В ходе данной лабораторной работы я научился численно интегрировать функцию с помощью метода Монте-Карло и нескольких его вариаций.